

VANTAGE

Sham Ganesh Thamarai Kannan
Bachelor of Software Engineering @ Monash University Malaysia

Submission Details

Deployed Application Link: <https://vantage-frontend-t028.onrender.com/>

Application Repository: <https://github.com/SGTK06/Vantage>

Demo Video: <https://youtu.be/yUsPgnfQG0g>

Plan & Approach

The goal of this project is to develop an application for SMEs to manage their invoices and track their expenses over time. This app eliminates the friction in managing invoices (manual data entry, bunch of spreadsheets that are archived but not used effectively, etc.).

Approach: Continuous Integration & Continuous Deployment

1. Plan the app functionality and list down all the features
2. Analyse alternatives and select a suitable tech stack
3. Initialize repository according to the chosen tech stack and setup CI/CD
4. Incrementally develop each feature in its dedicated branch and test it out before merging the branch into main.

Tools & Frameworks

This app has been built using React and FastAPI, with Supabase for authentication, database, file storage, and vector database. The app has been deployed on Render.

React (Frontend)	• Chosen for its flexibility, stability, and clear separation from backend code. • React frontend can be deployed independent of the backend.
FastAPI (Backend)	• Suitable for rapid prototyping, and is easy to deploy. • Clear separation from frontend code allows for extensive testing. • With Python as the backend language, it also supports any AI/ML related extensions and features
Supabase (Auth+Database)	• Handles authentication and is secure. • It is versatile as it can be used as a Postgres DB, Vector DB, and a files bucket.
Render (Deployment)	• Suitable for deploying both frontend and backend. Easy to prepare for deployment with one render.yaml file. • Compute limits are adequate given the requirements of this application.

Alternative: An alternative stack would be Next.js deployed on Vercel with Supabase. This one has not been chosen as the backend logic cannot be isolated and tested thoroughly. This would also make any future AI related extensions comparatively difficult

Developer Tools:

Development	Testing	Ideation (Brainstorming)
Antigravity	Codex	Claude

During the planning phase of the application, I used Claude for ideation and identifying what features are not offered by other competitor apps currently in the

market. I also used Claude to point out any potential flaws in my backend architecture before implementation. I used Antigravity for the first round of development primarily for the availability of Gemini 3.7 Flash, I then verified the backend logic in the code output (if it matches the given specification), tested the developed feature, then developed test cases to double check the logic using Codex. I also used Impeccable skill (<https://impeccable.style/>) for designing the UI of this application.

Software Architecture

The app first accepts PDF invoices from the user. It processed the PDF using **LlamaParse**. LlamaParse has been chosen over a standard LLM with a structured output parser because it is faster, more reliable, and fine-tuned well to avoid any unexpected errors. It can also run OCR on scanned invoices.

Then the items identified in the invoice are run through a **categorization-pipeline** where the item is allocated into one of the existing categories (through **semantic matching** by cosine similarity of item and category's embedding vectors), or if no good match is identified, as a fallback the pipeline asks **Gemma-4** to create a new category for this item. Gemma-4 has been chosen over a more powerful model like Gemini-3.5-Flash because it is sufficient for this trivial task. The embedding vector for the categories is obtained using the gemini-embedding-001 model and stored in Supabase.

Instead of embedding all the invoice items to representative vectors and then clustering into categories, this hybrid technique is faster and more efficient as the categorization is cached by storing directly in the database and then later queried for getting statistics immediately instead of re-grouping or running clustering algorithms for the purchased items often.

The app also supports **Natural-Language-Querying** to allow users to analyse their spending history over time by talking to an analyst agent, rather than stare at charts for insights. This is done so that new users do not feel daunted by the charts and can get insights validated by actual data as easily as possible. **Gemini-3.5-Flash-Lite** has been used as the model behind the analyst agent for its tool-calling capabilities and its has been chosen over more powerful models like Gemini-3.5-Pro because the tool calls ground the results produced by the model and hence the flash-lite model is sufficiently powerful to use computed results.

The Analyst agent is only given access to the **tools** which do the actual database querying and computation. These tools act as a mediator between the AI agent and the database to ensure that the agent does not tamper with the data in database, and only the computed final results are used in answering the user's query rather than rows of data, which reduces the chances of the model hallucinating answers.

Flowchart & Diagram

The flowchart for the core PDF parsing and item categorization pipeline is:



The analyst agent is a simple ReAct agent (Reasoning + Action) that uses statistical data to answer questions.

